Project Gringo — Argentinian Spanish Slack Bot

What

Slack bot that teaches Argentinian Spanish (Rioplatense) to 6-15 people through daily lessons, voice memo practice, AI conversation, and personalized feedback with long-term memory. Self-tuning — adapts based on user performance and feedback. Admin-controllable without redeployment.

Tech Stack

- **TypeScript + Node.js 18+**
- **@slack/bolt** — Slack bot framework
- **Claude Haiku** — tutor brain (Anthropic, $0.25/$1.25 per 1M tokens)
- **Deepgram Nova-3** — voice memo transcription ($0.0043/min)
- **Google Cloud TTS** — optional audio responses ($4-16/1M chars)
- **better-sqlite3** — database with WAL mode
- **node-cron** — scheduled content
- **vitest** — testing

Slack Channels

Channel	Purpose
`#daily-lesson`	Scheduled daily lesson: vocab + grammar + cultural note + listening exercise + voice prompt
`#charla-libre`	Open conversation and multi-turn dialogue simulations via voice memos
`#lunfardo-del-dia`	Daily slang word with etymology and usage
`#repaso`	SRS review sessions
`#desafios`	Pair practice and conversation challenges between users

Slash Commands

Command	Description
`/gringo help`	Channel guide and commands
`/gringo level`	Set or view your level (1-5)
`/conjugar <verb>`	Voseo conjugation table
`/vocab <word>`	AI word lookup with context
`/repaso`	Start SRS review session
`/charlar <scenario>`	Multi-turn dialogue simulation
`/shadow`	Shadowing/pronunciation exercise

`/desafio`	Pair practice challenge
`/feedback <message>`	Tell the bot what's working or not
`/admin curriculum`	View/edit lesson sequence (admin)
`/admin prompt <name>`	View/edit system prompts live (admin)
`/admin level <user> <n>`	Override user level (admin)
`/admin seed <topic>`	Add vocab/phrases to DB (admin)
`/admin schedule`	Change posting times, toggle features (admin)
`/admin feedback`	View error patterns and user feedback (admin)

Voice Memo Pipeline

```
User records voice memo in Slack
  → file_shared event → bot downloads audio
  → Deepgram Nova-3 transcribes (Spanish)
  → Claude Haiku analyzes (with user memory context)
  → Bot replies in-thread: corrections, feedback, new vocab
  → Errors logged, SRS cards created/updated
```

Cost per voice memo: ~$0.01-0.02

Interaction Flows

Daily Lesson (`#daily-lesson` , posted at scheduled time)

1. Cron triggers lesson generation
2. Claude generates a lesson based on curriculum position and group engagement data:
   - 3-5 new vocabulary words (with audio pronunciation via TTS)
   - A grammar point (voseo conjugation rule, sheísmo, etc.)
   - A cultural note (mate etiquette, Buenos Aires neighborhoods, etc.)
   - A listening comprehension audio clip
   - A voice prompt for users to respond to
3. Bot posts as rich Slack blocks
4. Users reply in-thread with voice memos
5. Bot transcribes, analyzes, and replies with personalized feedback referencing their history
6. SRS cards auto-created for new vocab

Listening Comprehension (part of daily lesson)

1. Bot generates a short audio clip via TTS — a sentence or mini-dialogue in Rioplatense Spanish at natural speed
2. Audio includes lunfardo, voseo, sheísmo appropriate to the group's average level
3. User must either:
   - Transcribe what they heard (tests listening accuracy)
   - Answer a comprehension question about the clip
4. Bot compares their transcription to the original, highlights what they missed

5. Progressively increases speed and complexity over time

Charla Libre (`#charla-libre` , anytime)

1. User sends a voice memo (or text) in the channel
2. Bot transcribes if voice
3. Bot responds in Rioplatense Spanish (using voseo, appropriate lunfardo for user's level)
4. Bot includes inline corrections if the user made errors
5. Conversation continues in-thread
6. New vocab encountered gets added to user's SRS deck

Dialogue Simulations (`/charlar <scenario>` , multi-turn)

1. User starts with `/charlar <scenario>` (e.g., "en el kiosco", "pidiendo un remís")
2. Bot responds with an audio message (TTS) as a character in the scenario
3. User responds with a voice memo
4. Bot responds to *that* with another audio message — building a back-and-forth
5. Continues for 5-10 turns, then bot provides a summary: corrections, new vocab, fluency notes
6. Each turn builds on the previous — bot may ask follow-up questions, change topics, throw in unexpected lunfardo

Shadowing Exercises (`/shadow` or part of daily lesson)

1. Bot plays a phrase via TTS at natural Rioplatense speed
2. User records themselves repeating it as closely as possible
3. Bot transcribes both the original and the user's version
4. AI compares: did they get the voseo right? Did they produce the sheísmo? Did they match the rhythm?
5. Feedback focuses on pronunciation patterns, not just word accuracy
6. Difficulty scales: short phrases → full sentences → rapid dialogue excerpts

Lunfardo del Día (`#lunfardo-del-dia` , daily)

1. Cron picks a lunfardo word the group hasn't seen yet
2. Posts: word, pronunciation, etymology (usually Italian origin), meaning, example sentence, cultural usage note
3. Voice prompt: "Usá esta palabra en una oración y grabá un audio"
4. Users respond with voice memos, bot provides feedback

SRS Review (`/repaso` or daily reminder)

1. User types `/repaso` or bot sends daily reminder
2. Bot checks due SRS cards for that user
3. Posts cards one at a time in a thread:
   - Vocab card: shows Spanish word → user responds with voice memo using it in a sentence
   - Conjugation card: shows infinitive + tense → user responds with vos form (text or voice)
   - Phrase card: shows English → user responds with Rioplatense translation
4. Bot scores response, updates SM-2 fields
5. Summary at end: cards reviewed, accuracy, streak

Pair Practice (`#desafios` , bot-facilitated human conversation)

1. Bot pairs two users (randomly or by level) and assigns a scenario:
   - "Uno de ustedes es mozo en un bodegón, el otro quiere pedir comida"
   - "Están en la cancha discutiendo quién es mejor: Messi o Maradona"
   - "Uno necesita indicaciones para llegar a San Telmo"

2. Bot creates a thread for the pair
3. Users voice-memo each other back and forth in the thread
4. Bot monitors the thread silently, transcribing each message
5. After the pair finishes (or after 10 exchanges), bot posts a summary:
    - Corrections for both users
    - New vocab each person used
    - Voseo consistency check
    - Suggestion for what to practice next
6. Zero API cost for the conversation itself — only costs are transcription + the summary analysis

---

Memory System

Three-layer architecture, all in SQLite:

Layer 1: Structured Data (automatic, every interaction)

Raw facts captured from every interaction — SRS scores, errors, vocab encounters, session history, participation rate. No AI needed, just SQL inserts.

Layer 2: Learner Profile (AI-generated, periodic)

Every week (or every 20 interactions), the bot reads Layer 1 data and asks Claude to generate/rewrite a ~200-300 word learner summary:

- Strengths and weaknesses
- Recurring error patterns
- Interests and preferred topics
- Level trajectory and recommendations
- Pronunciation tendencies (inferred from transcription patterns)

Stored in `user_memory` table. Gets **rewritten** each time (not appended), so it stays a fixed size.

Layer 3: Active Context (assembled per-interaction)

Every Claude call injects:

- Their learner profile from Layer 2 (~300 tokens)
- Last 5-10 errors from Layer 1 (~200 tokens)
- Current SRS stats (~100 tokens)
- The current interaction content

Total memory overhead: ~~600 tokens per call (~~$0.001 extra). Stays the same size regardless of how long the user has been learning.

Future upgrade: Mem0

If the summarization approach feels limiting, swap Layer 2 for self-hosted Mem0 (Docker: FastAPI + pgvector + Neo4j). The Layer 1 structured data and Layer 3 context assembly stay the same — only the memory extraction/retrieval changes.

Admin Controls & Self-Tuning

Admin Commands (`/admin` , restricted to admin Slack user IDs)

- **curriculum** — view and reorder the lesson plan sequence, skip topics, add new ones
- **prompt <name>** — view/edit any system prompt in real-time. Changes take effect immediately, no redeployment
- **level <user> <level>** — manually override a user's level
- **seed <topic>** — add new vocabulary, phrases, or lunfardo terms on the fly
- **schedule** — change daily lesson time, enable/disable channels or features
- **feedback** — view aggregated data: common errors across all users, user feedback messages, lesson engagement rates

User Feedback

- **Reactions on lessons** — thumbs up/down tracked on daily lessons and lunfardo posts. Bot learns which formats and topics resonate
- `/feedback <message>` — users can say "too easy", "more food vocab", "corrections feel harsh". Stored in DB
- **Post-charla prompt** — after dialogue simulations, bot asks "Was this useful? Too hard?" with button options

Self-Tuning (automatic)

The lesson generator prompt is injected with:

- Group-wide weak areas from error logs ("70% of users miss -ir conjugations → emphasize this week")
- Engagement data ("lunfardo lessons get 3x more voice memo responses than grammar drills → lean into lunfardo")
- Aggregated user feedback ("3 users said 'more food vocab' this week")
- Individual memory profiles for personalized follow-up even on shared lessons

Prompts in Database

All system prompts are seeded into the `system_prompts` table on first run from the default files in `src/llm/prompts/`. After that, the DB version is authoritative. `/admin prompt conversation` lets you tweak the porteño persona live. The files in `src/llm/prompts/` serve as defaults/fallbacks only.

What It Teaches

- **Voseo** — vos conjugations across all tenses (hablás, comés, vivís)
- **Lunfardo** — ~200 Buenos Aires slang terms with Italian etymology
- **Yeísmo/Sheísmo** — ll/y pronounced as "sh" (calle → cashe, playa → plasha)
- **Vesre** — syllable-flipping wordplay (café → feca, hotel → telo)
- **Cultural context** — mate etiquette, porteño customs, regional expressions

Per-User Levels (1-5)

Level	Vocab	Grammar	Lunfardo	Voice Prompts
1	Basic greetings, numbers, common words	Present tense voseo only	None yet	Simple (introduce yourself, order food)
2	Daily life vocab, food, transport	Present + past tense voseo	Basic (che, dale, boludo)	Descriptive (describe your day)
3	Work, emotions, opinions	All tenses + imperative	Common lunfardo	Situational (at the market, giving directions)

4	Abstract concepts, idioms	Subjunctive, conditionals	Full lunfardo + vesre	Complex (debate, tell a story)
5	Nuanced expression, humor	All grammar, natural flow	Deep lunfardo, wordplay	Free-form (joke, persuade, negotiate)

SRS (SM-2 Algorithm)

- Quality 0-5 (0-2 fail, 3 hard, 4 good, 5 easy)
- Standard SM-2 intervals with card-type modifiers:
  - Vocab: standard intervals
  - Conjugation: ×0.8 (more reinforcement needed)
  - Phrases: ×1.2 (more contextual/memorable)
- Auto-creates SRS cards when users encounter new vocab in lessons or conversations
- Voice memo responses scored by AI (checks if word was used correctly in context)

Database Tables

Table	Purpose
`users`	Slack user ID, level (1-5), XP, streak, last_practice, preferred difficulty
`vocabulary`	Spanish/English pairs with category, difficulty, examples, pronunciation, cultural notes
`conjugations`	Full voseo paradigms per verb per tense, with tú comparison
`srs_cards`	SM-2 fields (ease_factor, interval_days, repetitions, next_review) per user per content item
`review_log`	Individual review records for analytics
`conversation_threads`	Thread-based conversation tracking for charla-libre
`lesson_log`	Lessons posted and user participation
`user_vocab_encounters`	Which words each user has seen/practiced
`user_errors`	Categorized error log (grammar, vocab, conjugation, pronunciation) with timestamps
`user_memory`	AI-generated learner profile summaries, rewritten periodically
`system_prompts`	All system prompts stored in DB, editable via `/admin prompt`
`user_feedback`	User feedback messages and lesson reactions
`lesson_engagement`	Which lessons got voice memo responses and reactions

Content Data (seeded from JSON)

- ~500 core vocabulary entries
- ~200 lunfardo terms with etymology
- ~50 vesre pairs
- ~100 verb conjugation tables (all tenses)

- ~200 common phrases

Project Structure

```
src/
  app.ts                          # Entry point, Slack Bolt app
  config/
    env.ts                        # Environment config (req/opt pattern)
    types.ts
  handlers/
    voiceMemo.ts                  # Voice memo pipeline
    dailyLesson.ts                # Daily lesson generation
    lunfardoDelDia.ts             # Daily slang word
    charlaLibre.ts                # Open conversation
    repaso.ts                     # SRS review sessions
    commands.ts                   # Slash command router
    admin.ts                      # Admin commands
    desafios.ts                   # Pair practice
    listening.ts                  # Listening comprehension
    shadowing.ts                  # Shadowing exercises
  memory/
    userMemory.ts                 # Profile storage/retrieval
    contextBuilder.ts             # Per-interaction context assembly
    profileGenerator.ts           # Periodic AI summarization
  db.ts                           # SQLite singleton
  db/
    schema.sql
  errors/
    gringoError.ts
    formatUserFacingError.ts
  llm/
    client.ts                     # Anthropic client + chat()
    parseJsonFromLlm.ts
    prompts/                      # Default prompts (seeded to DB on first run)
      lessonGenerator.ts
      voiceAnalysis.ts
      conversation.ts
      conjugation.ts
      vocabulary.ts
      cultural.ts
  voice/
    transcribe.ts                 # Deepgram STT
    tts.ts                        # Google Cloud TTS
  content/
    vocabulary.ts
    conjugations.ts
    lunfardo.ts
    vesre.ts
    phrases.ts
    seed.ts
    data/
```

```
      vocab-core.json
      lunfardo.json
      vesre.json
      conjugation-tables.json
      phrases.json
  srs/
    scheduler.ts                  # SM-2 algorithm
    types.ts
    review.ts
  scheduler/
    cron.ts                       # Daily lessons, lunfardo, reminders
  observability/
    context.ts
  utils/
    logger.ts
    slackHelpers.ts
    levelAdapter.ts
```

Build Phases

Phase 1: Foundation

`app.ts`, `config/`, `errors/`, `db.ts` + `schema.sql`, `utils/`, basic Slack Bolt setup → Bot connects to Slack, responds to `/gringo help`

Phase 2: Voice Pipeline + Content

`voice/transcribe.ts`, `llm/client.ts`, `content/seed.ts`, `content/data/*.json` → Bot can receive voice memos, transcribe them, and has vocabulary data loaded

Phase 3: Daily Lessons

`handlers/dailyLesson.ts`, `handlers/lunfardoDelDia.ts`, `scheduler/cron.ts`, `llm/prompts/lessonGenerator.ts` → Bot posts daily lessons and lunfardo, processes voice memo responses with feedback

Phase 4: Charla Libre

`handlers/charlaLibre.ts`, `llm/prompts/conversation.ts`, `llm/prompts/voiceAnalysis.ts` → Open conversation practice and dialogue simulations with AI in Rioplatense Spanish

Phase 5: Spaced Repetition

`srs/scheduler.ts`, `srs/review.ts`, `handlers/repaso.ts` → Working SM-2 review system with voice memo support

Phase 6: Conversational Fluency

`handlers/listening.ts`, `handlers/shadowing.ts`, `handlers/desafios.ts`, TTS integration for audio prompts → Listening comprehension, shadowing exercises, pair practice

Phase 7: Admin & Self-Tuning

`handlers/admin.ts`, `system_prompts` table, `user_feedback` table, `lesson_engagement` tracking, `/feedback` command → Admin can edit prompts/curriculum/levels live, bot adapts lessons based on engagement and error patterns

Phase 8: Polish

Level adaptation, expanded content, embed formatting → Complete feature set

---

Verification

1. `npm run dev` — starts bot with tsx, connects to Slack
2. Test `/gringo help` → bot responds with channel guide
3. Send voice memo in `#charla-libre` → bot transcribes and responds in Rioplatense Spanish
4. Wait for daily lesson in `#daily-lesson` → lesson posts with voice prompt
5. Reply to lesson with voice memo → bot gives personalized feedback in-thread
6. Test `/repaso` → bot presents due SRS cards
7. Test `/conjugar hablar` → shows voseo conjugation table
8. Test `/charlar en el kiosco` → multi-turn dialogue with TTS audio responses
9. Test `/shadow` → plays phrase, user repeats, gets pronunciation feedback
10. Test pair practice in `#desafios` → bot pairs users, monitors, gives summary
11. Test `/admin prompt conversation` → view/edit the porteño persona prompt
12. Test `/feedback too easy` → feedback stored, reflected in next lesson generation
13. `npm test` → runs vitest suite (SRS algorithm, voice pipeline, content loading)

Cost Estimate (6-15 users)

Component	Per user/day	15 users/month
Deepgram Nova-3 STT (2 voice memos/day avg)	~$0.005	~$2.25
Claude Haiku (lesson + feedback + conversations)	~$0.06	~$27.00
Google Cloud TTS (optional audio responses)	~$0.01	~$4.50
Total	**~$0.075**	**~$34/month**

API Keys Required

- **Anthropic** — Claude Haiku (tutor, lessons, analysis, memory)
- **Deepgram** — Nova-3 (voice transcription)
- **Google Cloud** — TTS (optional, can skip initially)